

Subiecte Acadnet Interoperabilitate Software

Etapa Națională 2018

1. Broken Keyboard

Dennis R. este elev în clasa a 4-a și tocmai ce a primit prima sa tema la programare: să implementeze un program care să calculeze suma sau diferența a două cifre primite de la tastatură. La nici 2 minute după ce a început tema observă că i s-a stricat tastatura și nu mai poate scrie caracterul ‘;’. Determinat să facă tema în ciuda acestui mic impediment observă că acesta este un caracter destul de important și nu reușește să compileze fără el. Ca orice student care simte cum se apropie deadline-ul și nu reușește să termine tema întreabă un prieten, în acest caz pe fratele său mai mare, Ken T., student la Poli care are ceva experiență cu dezvoltarea unui sistem. Acesta acceptă provocarea de a termina programul fără niciun ‘;’.

Restricții: Implementați programul descris mai sus pornind de la scheletul de cod dat fără a folosi caracterul ‘;’ în cod.

Exemplu de executare program:

```
./broken-keyboard
3+5
8
`
`

./broken-keyboard
2-6
-4
`
`

./broken-keyboard
9+5
14
```

Codul sursă:

```
#include<stdio.h>

void main(){
    int rez = 0;
}
```

Soluția:

```
#include<stdio.h>

void main(){
```

```

int rez = 0;
if (rez = fgetc(stdin)) {
    if (fgetc(stdin) == '-') {
        if (!printf("%d\n", rez - fgetc(stdin))) {}
    } else {
        if (!printf("%d\n", rez + fgetc(stdin) - 48*2)) {}
    }
}
}

```

2. Gigel vs Gigelinho

Gigel se gândește cum ar suna numele lui în portugheză. Deoarece îi place foarte mult cum sună **Gigelinho**, el a scris un program **C++** care să illustreze faptul că varianta în portugheză este de două ori mai placută.

El a scris un program care ar trebui să citească un număr **gigel** din fișierul **input.txt** și să afișeze pe ecran numărul **dublat** (adică plăcere dublă).

Din păcate programul **nu compilează**. Trebuie să îl faceți să meargă.

NU aveți voie să ștergeți linii din fișier.

NU aveți voie să modificați orice linie din fișier.

Singurele linii care pot fi modificate sunt liniile **21** și **25**.

Examples:

Input in input.txt	Output
1	2

Input in input.txt	Output
5	10

Input file name: input.txt

Time limit: 1 s

Real time limite: 5 s

Memory limit: 64 M

Codul sursă:

```

#include <iostream>
#include <cstdio>
using namespace std;

```

```

int main() {
    // următoarea linie oferă posibilitatea de a citi din fișierul "input.txt"
    // folosind în continuare cin!
    freopen("input.txt", "r", stdin); // este corectă această linie, nu trebuie să modifici

    // Observați liniile 10-15. Nu trebuie să le modificați.
    int gigel;
    gigel = 1, 2, 3; // gigel == ?
    ++gigel = 2;     // gigel == ?
    ++gigel = 3;     // gigel == ?

    // Acum să terminăm cu prostiile și să îl citim pe gigel din
    // fișier. Gigel este (un) întreg.

    cin >> gigel;

    // Aveți voie să modificați linia 21.
    int enumeration = 1, 2, 3;
    ++enumeration;

    // Aveți voie să modificați linia 25.
    int a = 1, b = 2, c = 3, d = 4, e = 5, f = 6;

    int foo, bar;
    foo = ++f, e - b, c - a;
    bar = (a + b, ++b + ++c - --a + a++, f);

    cout << (foo + bar) << "\n";

    return 0;
}

```

Soluția:

```

#include <iostream>
#include <cstdio>
using namespace std;

int main() {
    // următoarea linie oferă posibilitatea de a citi din fișierul "input.txt"
    // folosind în continuare cin!
    freopen("input.txt", "r", stdin); // este corectă această linie, nu trebuie să modifici

    // Observați liniile 10-15. Nu trebuie să le modificați.
    int gigel;
    gigel = 1, 2, 3; // gigel == ?
    ++gigel = 2;     // gigel == ?

```

```

++gigel = 3;      // gigel == ?

// Acum să terminăm cu prostiile și să îl citim pe gigel din
// fișier. Gigel este (un) întreg.
cin >> gigel;

// Aveți voie să modificați linia 21.
int enumeration = 1, junk = 2, other_junk = 3;
++enumeration;

// Aveți voie să modificați linia 25.
int a = 1, b = 2, c = 3, d = 4, e = 5, f = gigel - 1;

int foo, bar;
foo = ++f, e - b, c - a;
bar = (a + b, ++b + ++c - --a + a++, f);

cout << (foo + bar) << "\n";

return 0;
}

```

3. Limits

Gigel rezolvă probleme la matematică pentru examenul de Bacalaureat și întâmpină o problemă în calculul unei limite.

$\lim_{x \rightarrow 0} (1 - \cos(x)) / (x^2)$

El se gândește că ar obține rapid un rezultat dacă ar scrie un program care să calculeze această limită. Programul lui, limits.cpp, citește un număr values din fișierul input.txt și calculează valoarea limitei pentru

$x = 1 / (2^{\text{values}})$

Din păcate programul nu se execută corect. Trebuie să îl faceți să meargă.

Singurele linii care pot fi modificate sunt cele care formează bucla while.

Examples:

Input in input.txt	Output
1	0.5 0.48967

Input in input.txt	Output
40	9.09495e-13 0.5

Input in input.txt	Output
--------------------	--------

Input file name: input.txt

Time limit: 1 s

Real time limite: 5 s

Memory limit: 64 M

Codul sursă:

```
#include <cmath>
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    // următoarea linie oferă posibilitatea de a citi din fișierul "input.txt"
    // folosind în continuare cin!
    freopen("input.txt", "r", stdin); // este corectă această linie, nu trebuie
    să modifichi aici!

    int values;
    cin >> values;

    double x = 1.01, limit;

    // trebuie sa modifichi aceasta bucla
    while (values --> 0) {
        x = x / 2;
        limit = (1 - cos(x)) / (x * x);
    }
    // nu modifica dupa aceasta linie

    cout << x << " " << limit << "\n";

    return 0;
}
```

Soluția:

```
#include <cmath>
#include <cstdio>
#include <iostream>

using namespace std;
```

```

int main()
{
    // următoarea linie oferă posibilitatea de a citi din fișierul "input.txt"
    // folosind în continuare cin!
    freopen("input.txt", "r", stdin); // este corectă această linie, nu trebuie
    să modifichi aici!

    int values;
    cin >> values;

    double x = 1.01, limit;

    // trebuie să modifichi această buclă
    while (values --> 0) {
        x = x / 2;
        double half = x / 2;
        double halfSine = sin(half);
        limit = halfSine / half;
        limit = limit * limit / 2;
    }
    // nu modifica după această linie

    cout << x << " " << limit << "\n";

    return 0;
}

```

4. Numere Prime

Gigel a început să învețe programare acum câteva luni și si-a propus să scrie un program care să îi spună câte numere prime sunt mai mici decât numărul **n** (citit de la tastatură). Gigel vrea de asemenea să calculeze suma acestor numere prime.

El a scris un program simplu care ar trebui să facă asta, dar din păcate, programul nu merge. De asemenea, la compilare, Gigel primește multe warning-uri. Ajutați-l pe Gigel să-și rezolve warning-urile și să își corecteze programul.

Exemplu input: 10

Exemplu output: 17 4

Explicatie: Numerele prime $\leq$ cu 10 sunt 2, 3, 5, 7. Suma lor este 17, numărul lor este 4

Codul sursă:

```

#include <iostream>
#include <cstdio>
#include <cstring>
#include <cstdlib>

bool is_prime(unsigned long long x) {
    if (x == 2) return true;

    for (long long i = 3; i * i <= x; i += (4/2))
        if (x % i == 0)
            return false;

    return true;
}

int main() {
    unsigned long long n, count, sum;
    char citire[100];
    gets(citire);
    n = atoi(citire);

    count++;
    sum += 2;

    for (int i = 3; i <= n; i += (6/3)) {
        if (is_prime(i))
            count++;
            sum += i;
    }

    std::cout << sum << " " << count << std::endl;
}

```

Soluția:

```

#include <iostream>
#include <cstdio>
#include <cstring>
#include <cstdlib>

bool is_prime(long long x) {
    if (x == 2) return true;

    for (long long i = 3; i * i <= x; i += 2)
        if (x % i == 0)
            return false;
}

```

```

        return true;
    }

    int main() {
        long long n, count, sum;
        count = 0;
        sum = 0;
        std::cin >> n;

        count++;
        sum += 2;

        for (int i = 3; i <= n; i += 2) {
            if (is_prime(i)) {
                count++;
                sum += i;
            }
        }

        std::cout << sum << " " << count << std::endl;
    }

```

5. Problema lui Gigel

Gigel se gândește la șirul Fibonacci, în care pozițiile din șir sunt numerotate cu **1, 2, 3, 4, 5, ...** iar termenii sunt **1, 2, 3, 5, 8, 13, 21, ...**

El a scris un program care ar trebui să citească un număr **n** din fișierul **input.txt** și să afișeze pe ecran al n-lea termen din șir.

Din păcate programul **nu compilează**. Trebuie să îl faceți să meargă. Rezolvarea se va face prin modificarea unui număr minim de linii de cod. Se considera linie modificată orice linie stearsă, adăugată sau alterată de voi.

Examples:

Input in input.txt	Output
5	8

Input file name: input.txt

Time limit: 1 s

Real time limite: 5 s

Memory limit: 64 M

Codul sursă:

```

#include <iostream>
#include <cstdio>

int fibbo(int n);

int main() {
    // următoarea linie oferă posibilitatea de a citi din fișierul "input.txt"
    // folosind în continuare cin!
    freopen("input.txt", "r", stdin); // este corectă această linie, nu trebuie să modifi

    int n;
    cin >> n;
    cout << fibbo(n) << "\n";

    return 0;
}

int fibbo(int n) {
    if (n == 1) return 1;
    if (n == 2) return 2;
    if (n == 3) return 3;

    int a = 1; //fibbo[1]
    int b = 2; //fibbo[2]
    int c = 3; //fibbo[3]

    // fibbo[i] = fibb[i-1] + fibbo[i-2], i > 3
    for (int i = 4; i <= n; ++i) {
        int f = b + c;
        b = c;
        c = f;
    }

    return c;
}

```

Soluția:

```

#include <iostream>
#include <cstdio>
using namespace std;

int fibbo(int n);

int main() {
    // următoarea linie oferă posibilitatea de a citi din fișierul "input.txt"
    // folosind în continuare cin!

```

```

    freopen("input.txt", "r", stdin); // este corectă această linie, nu trebuie să modifici

    int n;
    cin >> n;
    cout << fibbo(n) << "\n";

    return 0;
}

int fibbo(int n) {
    if (n == 1) return 1;
    if (n == 2) return 2;
    if (n == 3) return 3;

    int a = 1; //fibbo[1]
    int b = 2; //fibbo[2]
    int c = 3; //fibbo[3]

    // fibbo[i] = fibb[i-1] + fibbo[i-2], i > 3
    for (int i = 4; i <= n; ++i) {
        int f = b + c;
        b = c;
        c = f;
    }

    return c;
}

```